



RAPPORT SAE

Coupe de France robotique



université
PARIS-SACLAY

IUT DE CACHAN

16 JANVIER 2023

ROBINSAN
Kiritheepan

Table des matières

1	Le comptage des points :	2
1.1	Faire des gâteaux :	2
1.2	Mettre la cerise sur le gâteau :	2
1.3	Ranger les cerises dans le panier :	2
1.4	Mettre les roues dans le plat :	2
1.5	Se déguiser pour faire la fête :	3
1.6	Calculer l'addition :	3
1.7	Les pénalités :	3
1.8	Simulation de comptage de points :	4
2	Carte herkulex :	5
2.1	Photo de la carte :	5
3	Conception du schéma électronique :	6
3.1	Library :	6
3.2	Le schéma :	6
3.3	Le PCP :	7
3.4	Photo du routage :	7
4	Soudage de la carte :	8
5	Teste de la carte :	9
5.1	Programme test herkulex :	9
5.2	Programme test du moteur :	10
5.3	Programme test capteur :	11
5.4	Programme test BusCAN :	11
6	Conclusion :	13

1 Le comptage des points :

La coupe de France de robotique est une compétition comme chaque compétition le comptage des points sont important. A la fin des 100 secondes, les robots doivent s'arrêter et éteindre l'ensemble de leurs actionneurs. Il est autorisé de conserver les afficheurs dynamiques allumés.

Le décompte des points sont réalisés par les arbitres, Si les équipes ne sont pas d'accord, elles en réfèrent calmement aux arbitres. Les robots restent en place tant que le litige n'est pas résolu. Les décisions d'arbitrage sont sans appel. Les arbitres peuvent décider de refaire le match ou de prononcer la fin d'un match avant les 100 seconde si les deux équipes sont en accord.

Le comptage des points se déroule par plusieurs étapes. En effet, l'arbitre prend en compte les points réalisés par les robots durant le match, les points d'estimations du score, les pénalités et les points bonus pour le score final.

1.1 Faire des gâteaux :

Pour un gâteau valide, il doit être composé d'au minimum 1 couche, et au maximum de 3 couches. Un gâteau peut être valide pour n'importe quel type de couche, On aura **1 point par étage** dans le gâteau. Mais si le gâteau respecte la recette légendaire (Glaçage + Crème + Génoise), On aura **4 points supplémentaires**. Une aire de dépôt ne pourra pas accueillir plus de 3 gâteaux et Un gâteau toujours contrôlé par un robot à l'issue du match ne sera pas compté.

1.2 Mettre la cerise sur le gâteau :

Pour qu'une cerise nous rapporte des points, elle doit être présente sur le dernier étage d'un gâteau valide et une seule cerise sera comptée par gâteau, Donc cela, nous rapporte **3 points pour chaque cerise posée sur un gâteau valide**.

1.3 Ranger les cerises dans le panier :

Si l'équipe dépose un panier pendant le temps de préparation a **5 points**, chaque cerise dans le panier compte **1 point** et si le panier réalise un comptage de la quantité cerise dans le panier Il aura **5 points supplémentaires** (le comptage est juste et supérieur à 0).

1.4 Mettre les roues dans le plat :

Les robots de l'équipe sont dans leurs aires de dépôt c'est à dire on aura une aire de dépôt réserve pour les robots (si 2 robots, les deux doit être dans le même) cela nous rapporte **15 points**

1.5 Se déguiser pour faire la fête :

Si le robot se déguise à la fin du match cela nous rapporte **5 points**, pour cela, le déguisement doit changer la couleur ou l'aspect du robot (si plusieurs robots, un seul suffit) et il doit avoir 50% du périmètre du robot avec un hauteur max de 35 cm (au moins 15 cm de hauteur). Le déguisement doit être contenue dans les robots du début à la fin du match et l'action doit être réalisée avant la fin du match, et après cela le robot déguisé doit s'arrêter et éteindre l'ensemble de leurs actionneurs.

1.6 Calculer l'addition :

Estimation du score permet d'avoir des points via le calcul suivante :

$$\text{Bonus} = 20 \text{ points} - \text{Ecart}$$

Ou, **Ecart = le score fait par l'équipe durant le match - le score estimé par l'équipe**

1.7 Les pénalités :

- perte de pièce ou d'élément d'un robot sur l'aire de jeu : **perte de 20 points.**
- dégradation de la table ou d'un élément de jeu : **perte de 30 points.**
- système d'évitement non fonctionnel : **perte de 30 points.**
- faux départ : **perte de 50 points.**
- le robot continue de bouger à la fin du temps imparti : **perte de 50 points.**
- temps de préparation excessif : **perte de 50 points.**
- le robot change de zone de départ après 3 minutes de préparation : **perte de 50 points.**
- comportement non fair-play : **perte de 50 à 100 points.**
- sur décisions de l'arbitrage : **perte de 50 à 100 points.**
- sur décisions de l'organisation : **perte de 50 à 100 points.**

Certaines actions peuvent entraîner **un forfait de l'équipe** ou **la disqualification de l'équipe de la compétition** (Par exemple, aucun robot ne sort de sa zone de départ ou dégradation volontaire de robot appartenant à d'autres équipes).

1.8 Simulation de comptage de points :

Scenario :

A la fin d'un match,

- 1 disque dans un Aire de dépose. **(1 point)**
- 3 disque superposer dans un Aire de dépose avec un cerise sur le dernier étage.

(3 points car 3 disques + 3 points pour la cerise = 6)

- 3 disque dans un Aire de dépose (recette légendaire).

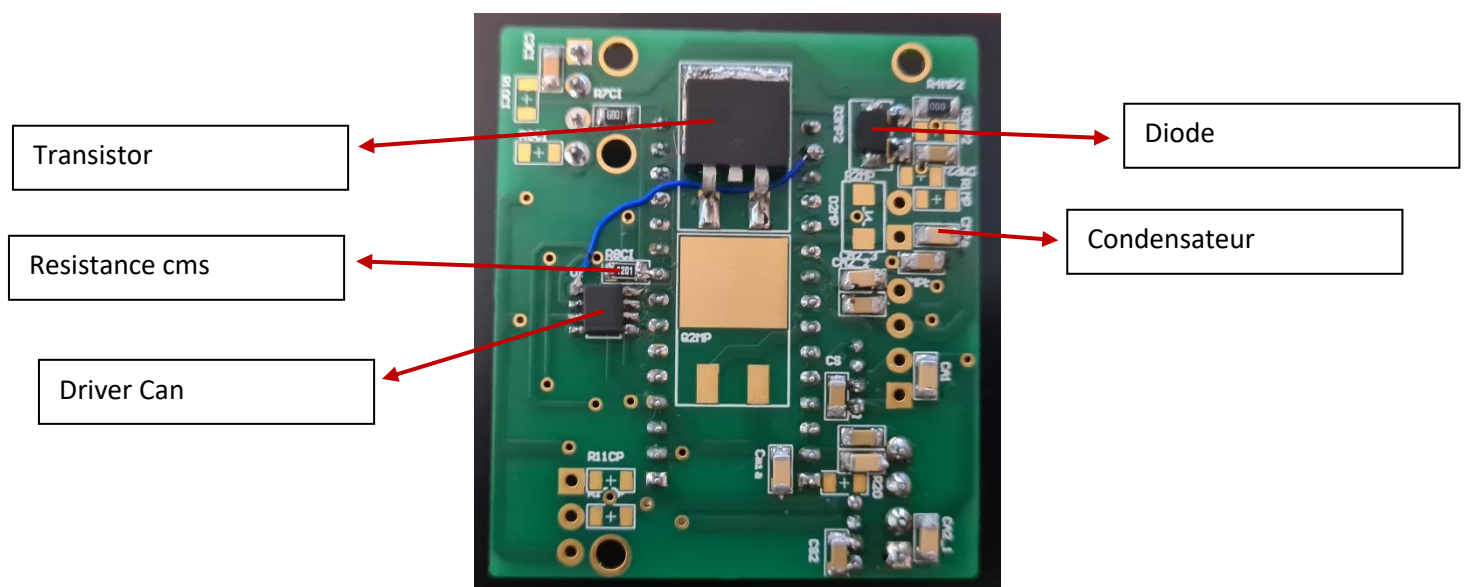
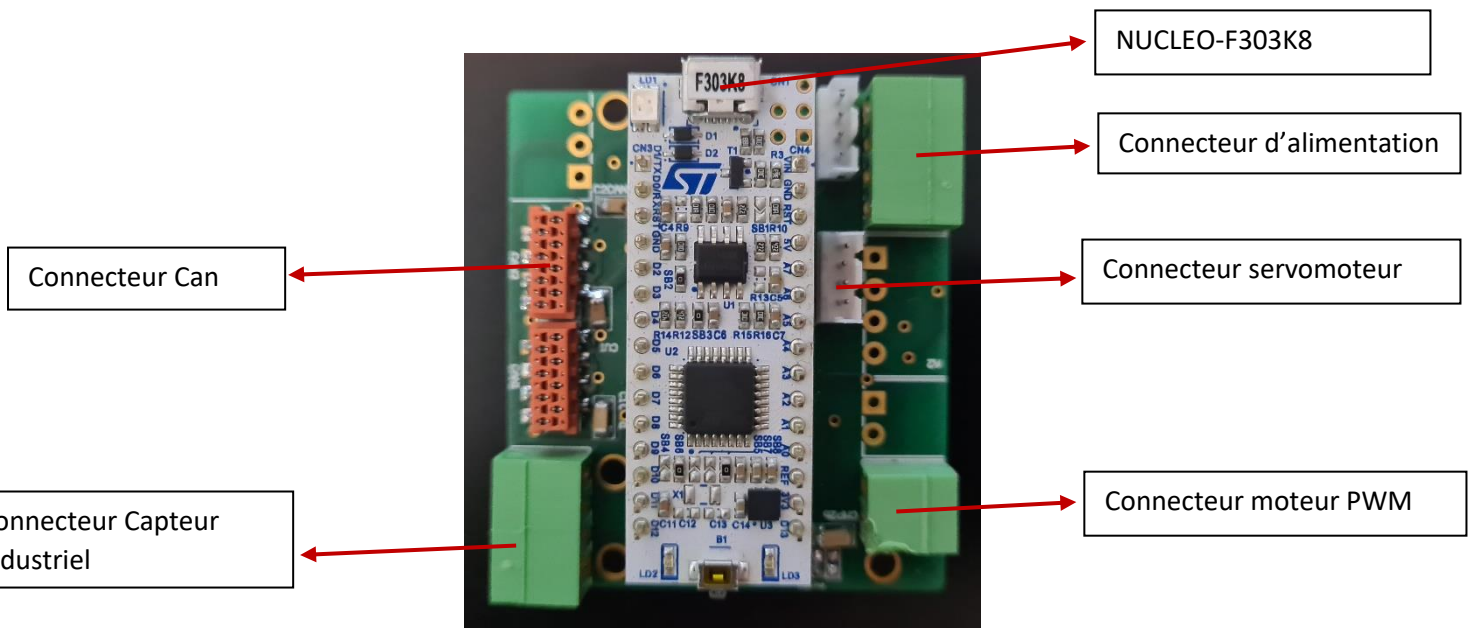
(3 points car 3 disques + 4 points recette légendaire = 7)

Les robots sont à l'arrêt dans la même aire de dépose **(15 points)** et l'un des robots se déguise juste avant l'arrêt **(5 points)**, il y a un panier **(5 points)** avec 4 cerises **(4 points)** mais sans comptage **(0 point)**. Donc, on a **un score de 43**. On a estimé le score a 37 points, Donc, (Bonus = 20 points – Ecart) et Ecart = score fin math – estimation, d'où **Ecart = 6**, Donc, On a **un bonus de 14**. L'arbitre n'a observé aucune pénalité, Ainsi, on a **score de 57 points**.

2 Carte herkulex :

Dans le cadre du projet nous avons remarqué la nécessité d'avoir une mini carte modulable à plusieurs situations. Cette carte devait être sous mbed, pour cela, on a utilisé un microcontrôleur NUCLEO-F303K8. La carte doit contrôler deux servomoteurs, un capteur industriel, deux sorties moteurs pwm et un capteur de pression. Pour cela, on a utilisé les archives du crac afin d'analyser les anciens circuits électroniques et on a effectué des recherches sur les composants.

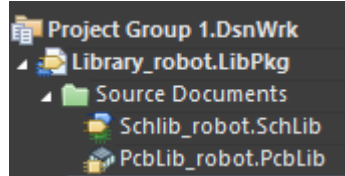
2.1 Photo de la carte :



3 Conception du schéma électronique :

3.1 Library :

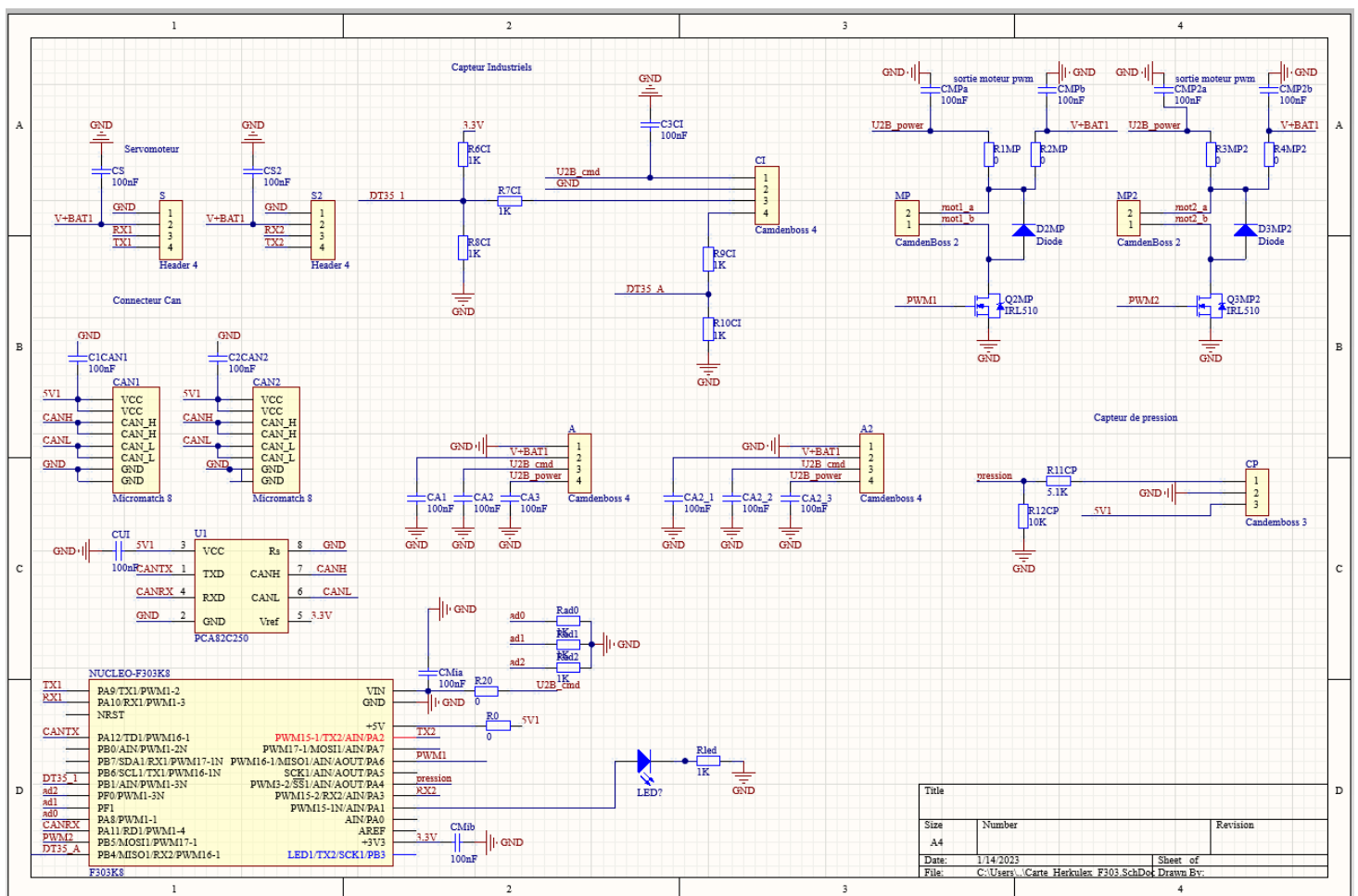
On crée deux Library, SchLib une librairie qui contient les composant des schémas et PcbLib qui contient les composant vue en PCP et en 3D pour cela j'ai utilisé le libraire geei1 pour sélectionner les composant qui m'intéresse.



3.2 Le schéma :

Après avoir, chercher les anciens schémas électroniques du crac, on a fait une analyse pour voir quelle porte du microcontrôleur correspond sur NUCLEO-F303K8. Donc pour cela on a utilisé le site mbed pour aller voir le schéma du microcontrôleur (les PINS) et en suite on a repris les anciens schémas sur Altium pour l'adapter. La difficulté était de comprendre les différents composants à utiliser et les liens entre le composant. Par exemple fallait comprendre qu'on a un driver can par microcontrôleur ou bien associer les pin du microcontrôleur selon le type de sortie ou d'entrée (RT, TX , PWM ,5V et 3V...etc).

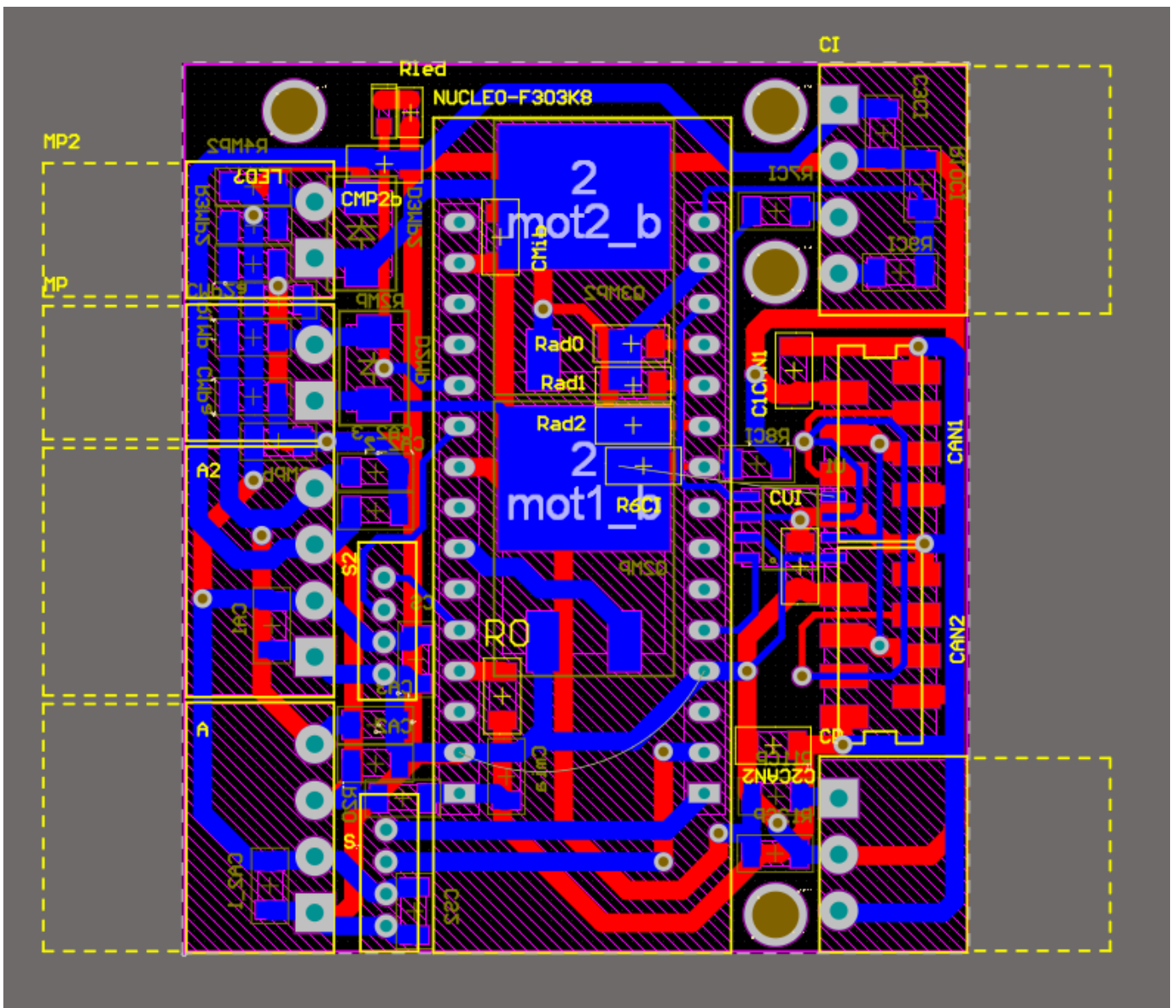
Schéma de la carte (Sur Altium) :



3.3 Le PCP :

Pour la création du PCB, fallait effectuer une capture du schéma c'est-à-dire fallait transférer les information SchDoc vers le PcbDoc nouvellement créé, et puis cliquez sur Design. Une boîte de dialogue Ordre de modification technique (ECO) s'ouvre, listant tous les composants et les nœuds du schématique et une fois valider les modifications, (si les éléments sont verts), puis appuyer sur exécuter les modifications. Ensuite faut définir les règles de conception, les taille des routages et des trous de perçage. Puis c'est l'étape, de positionnement des composant pour cela faut essayer de le mettre de façon logique et bien gérer l'espace puisque qu'on veut un mini carte (Nous avons placer les composant qui se correspond, ensemble, de façon à avoir un chemin logique pour le routage). Enfin, fallait router les pistes, il est important de bien sélectionner la taille des files.

3.4 Photo du routage :



On peut voir, sur cette photo que les composant sont placer de façon logique afin de crée des routages en ligne verticale et horizontale.

Photo du vue 3D de la carte :

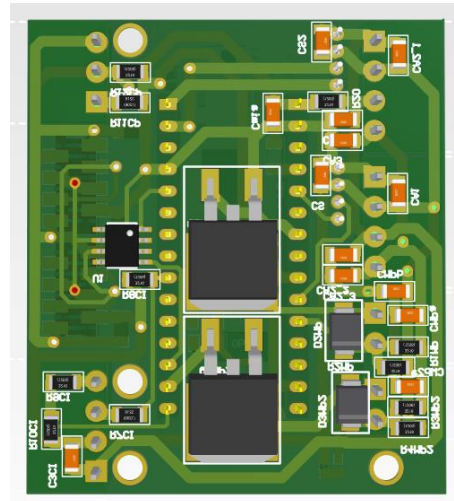
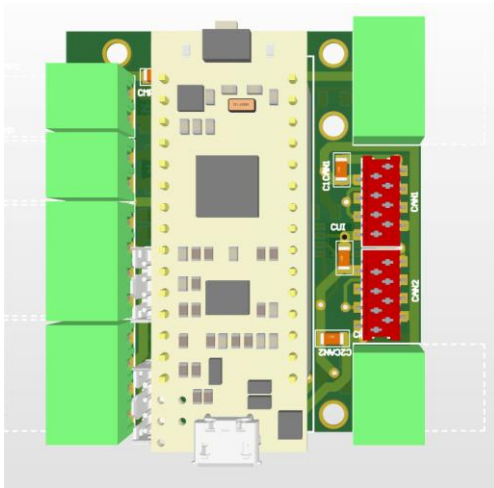
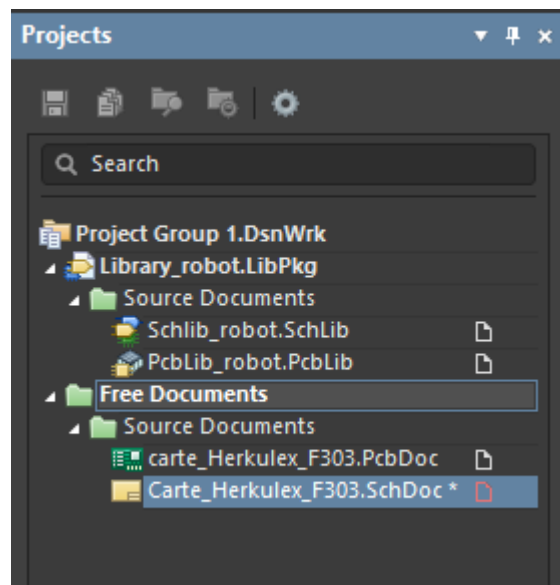


Photo de mes fichier Altium :



4 Soudage de la carte :

Une fois qu'on a reçu nos cartes PCB de l'entreprise responsable de réaliser, on a commencé à souder les composants pour cela fallait regarder les références des composants cms et la taille (par exemple on a utilisé des résistance cms 1206 et condensateur cms 1206). Il est très difficile de souder des petits composants donc, pour cela, il fallait réaliser un test d'isolement au fur et à mesure qu'on soude un composant c'est-à-dire regarder si le courant passe bien, si il n'y a pas de court-circuit, pour cela, on a utilisé le multimètre. Sur cette carte tous les composants n'ont pas été soudés puisqu'il est modulable à n'importe quelle situation en choisissant les fonctionnalités qu'on effectue avec la carte (par exemple on n'a pas soudé le capteur pression car on n'a pas besoin pour l'instant).

Après le soudage, on remarquer un léger problème au niveau du busCAN puisque on avait deux connecteurs can très proche, donc, impossible de brocher les deux en même temps. Par conséquent, il faudra penser à écarter un peu lors des prochains soudages.

5 Teste de la carte :

Cette partie consister à utiliser le microcontrôleur afin de tester le bon fonctionnement de la carte pour cela on a réalisé quelque programme test.

5.1 Programme test herkulex :

Pour tester les servomoteurs on fait varier entre deux position 290 et 800, pour cela fallait utiliser le mon identifiant du servomoteur donner les bons pins du microcontrôleur.

Code:

```
/* mbed Microcontroller Library
 * Copyright (c) 2019 ARM Limited
 * SPDX-License-Identifier: Apache-2.0
 */

#include "mbed.h"
#include "herkulex.h"

// set serial port and baudrate, (mbed <-> HerculexX)
Herkulex sv(PB_6, PB_7, 115200);

// Create a DigitalOutput object to toggle an LED whenever data is received.
static DigitalOut led(LED1);

void wait(int s)
{
    wait_us( s * 1000000);
}

unsigned int id = 0x5;

int main()
{
    sv.clear(id);

    // sv.positionControl(id,1, 30, GLED_ON);
    printf("Status %d\n", sv.getStatus(id));
    printf("Current position %d\n", sv.getPos(id));
    printf("Status %d\n", sv.getStatus(id));
    printf("Current position %d\n", sv.getPos(id));

    sv.setTorque(id, TORQUE_ON);

    while(1)
    {
        sv.positionControl(id, 290, 30, GLED_ON);
        printf("Status %d\n", sv.getStatus(id));
        printf("Current position %d\n", sv.getPos(id));
    }
}
```

```

        wait(10);

        sv.positionControl(id, 800, 30, GLED_ON);
        printf("Status %d\n", sv.getStatus(id));
        printf("Current position %d\n", sv.getPos(id));

        wait(10);
    }
}

```

5.2 Programme test du moteur :

On faisait varier le rapport cyclique du pmw, on accélère le moteur au fur a mesure du temps.

Code :

```

/* mbed Microcontroller Library
 * Copyright (c) 2019 ARM Limited
 * SPDX-License-Identifier: Apache-2.0
 */

#include "mbed.h"

#define WAIT_TIME_MS 500
DigitalOut led1(LED1);
PwmOut moteur(PB_5);

int main() {
    // specify period first, then everything else
    moteur.period(0.001f); // 4 second period

    //moteur.pulsewidth(2);
    while(1) {
        moteur.write(0.0f); // 50% duty cycle
        ThisThread::sleep_for(1s);
        moteur.write(0.10f); // 50% duty cycle
        ThisThread::sleep_for(1s);
        moteur.write(0.20f); // 50% duty cycle
        ThisThread::sleep_for(1s);
        moteur.write(0.50f);
        ThisThread::sleep_for(5s);
        moteur.write(0.80f);
        ThisThread::sleep_for(1s);
        moteur.write(0.100f);
        ThisThread::sleep_for(1s);
    }
}

```

5.3 Programme test capteur :

On utilise un capteur PNP pour tester si le capteur détecte un objet à une certaine distance. Donner, pour cela, j'ai soudé deux résistances avec un pont diviseur de tension pour avoir 3.2V en entrée du microcontrôleur à partir d'une entrée 21V.

Code 1 :

```
/* mbed Microcontroller Library
 * Copyright (c) 2019 ARM Limited
 * SPDX-License-Identifier: Apache-2.0
 */

#include "mbed.h"
#include <stdio>

#define WAIT_TIME_MS 500

AnalogIn capteur (PF_8);

int main()
{
    float capteur_valeur;

    while (true)
    {
        capteur_valeur = capteur.read();
        printf("capteur %f " "\n", capteur_valeur );
    }
}
```

5.4 Programme test BusCAN :

Le BusCAN permet la communication entre nos cartes dans le robot, pour tester cela, on a utilisé une autre carte (avec un encodeur), On envoie une requête avec le busCAN et on regarde s'il y a une réponse. Cela permet de tester l'envoi et la réception du message CAN. Sur le programme il faut indiquer l'id de la trame CAN pour la requête l'id est 0x7A0 et pour la réception c'est 0x7A1.

Code :

```
#include "mbed.h"
#include "rtos.h"
#include "Thread.h"

#define MAIN_SLEEP_TIME 200ms
CAN busCAN (PA_11, PA_12, 1000000);

CANMessage Rq_Msg (0x7A0, CANStandard);
```

```

CANMessage Rx_Msg;

EventFlags myFlags;
void CAN_ISR (void) {
    busCAN.read (Rx_Msg, 0);
    myFlags.set (1);
}

int main () {

    busCAN.frequency(1000000);
    busCAN.attach (&CAN_ISR); //IrqType::RxIrq
    while (true){
        busCAN.write (CANMessage(0x7A0, CANStandard));
        rtos::ThisThread::sleep_for (MAIN_SLEEP_TIME);
        for(int i = 0; i < 8; i++)
        {
            printf("TesT = %d\n",Rx_Msg.data[i]);
        }
    }
}
//id envoi 7A0 id reception 7A1

```

Code 2 :

Permet seulement la réception du valeur du bargraphe.

```

#include "mbed.h"
#include "ThisThread.h"

unsigned char data[2], test_char;
unsigned short id = 0x7B0, test_short;

typedef union {unsigned short mot; unsigned char octet[2];} T_bar;
T_bar bargraf;

CANMessage bar (id, data, 2, CANData, CANStandard);
RawCAN myCAN (PD_0, PD_1,1000000);

void fct_call (int ii);

int main()
{
    //bar.id = 0x7B0;
    while (true) {
        for(int i = 0; i<9 ; i++){
            fct_call(i);
        }
        for(int i = 9; i>0 ; i--){

```

```

        fct_call(i);
    }

}

void fct_call (int ii)
{
    bargraf.mot = 1 << ii;
    //test_short = bargraf.mot;
    for(int j = 0; j<2 ; j++){
        bar.data[1-j] = bargraf.octet[j];
    }
    myCAN.write(bar);
    ThisThread::sleep_for(20ms);
}

```

6 Conclusion :

On est plutôt satisfait de la mini-carte, on a remarqué aucun problème à part le fait de devoir écarter un peu les connecteur Can quand on le soude. Maintenant il faudra souder d'autre carte et voir comment le bien l'utiliser dans le robot.